

Image Classification

CS4346 – Spring

Ryan Ashworth

Introduction

This project requires coding and implementing Naive Bayes and Perceptron algorithms on a supplied data set of digit and face images. Each algorithm is trained using two user-defined feature extraction methods and then tested on a separate test dataset. The results of the test runs are then compared and analyzed.

Part 1: Data Processing

The original data for the digit and face datasets are divided into training, validation, and testing sets, with each set containing a file of representative images and a file of corresponding label values. The data in the image files are stored as lines of plain text characters. Each image must be separated from the full file and then matched to its corresponding value from the label file. This process is handled in *data_extraction.py*.

First, an empty vector is created to contain the completed labeled image data. The label value file is then read by splitting each line into a separate value, removing the newline characters, and storing the results in a vector in order. This gives a list of label values in the same order that the images appear in the image file. The next step is to read the image data using the same split-line process so that each line is parsed as a separate entry. Using the length of one line of image data gives the *image_width* value, and dividing the total number of lines in the image file by the length of the label-value list gives the *image_height* value that is used throughout the rest of the program.

The image data is then processed in groups of *image_height* lines, which allows each image to be separated from the full file. The characters, including blank spaces, from each line of the image are placed into a single continuous list. This list is then combined with its corresponding label value by tracking which image is currently being processed and using that index to attach the correct label.

Each of these labeled lists is combined into a *labeled_image_vector* of raw data that is used in the later stages of the program. All training, validation, and testing files are initially processed in this same manner.

Part 2: Feature Extraction from Data

Two methods of feature extraction are built into the program.

The first method, *raw_pixel_feature_extraction.py*, uses the labeled image data vector created during data processing. It follows a simple process of traversing each character in

the image representation and recording a *0* if the character is a blank space or a *1* if any other character appears. This produces a new vector, *raw_pixel_feature_vector*, of the same length as the original flattened image data, but with a binary representation of the image.

The second feature extraction method, *sliding_window_feature_extraction.py*, begins by running *raw_pixel_feature_extraction.py* to obtain the binary representation of the image data. It then loops through each image and counts how many times a *1* appears within a sliding 3x3 window. If there are more than three *1*s in the window, a value of *1* is assigned to that feature; otherwise, a *0* is assigned. This produces a new binary representation of the data that looks for patterns within a local region of each image rather than relying only on individual pixel values.

The sliding window is implemented on the flattened one-dimensional image representation by using the known image height and width to simulate movement across the image as if it were still two-dimensional. A loop is created that runs from 0 to (*image_height* - *window_size* + 1), increasing by the designated slide amount each time. A second loop does the same for the image width. This setup allows each valid window position to be processed without going out of bounds. For each window, the row and column positions are used to calculate the corresponding index values in the flattened image vector. The values inside the window are then counted, converted into a final *1* or *0* based on the threshold rule, and appended to the completed *sliding_window_feature_vector* along with the corresponding label value.

Part 3a: Implement Classification Algorithms – Naive Bayes

In *naive_bayes_train.py*, the Naive Bayes training algorithm begins by identifying the image type and feature type. This determines how many possible label values exist and which feature extraction function should be run. Using the training percentage argument, a *picture_data* vector is built by randomly selecting the corresponding number of images from the full training dataset. The number of feature values to process is obtained from the length of the data portion of the first vector entry. A list of label-value counts is initialized with a value of *0* for each possible label. In addition, a 2D array is created to collect the feature counts for each label value, with all entries initially set to *0*.

Training begins by looping through each image in *picture_data*. For each image, the count for its label value is incremented. The feature data is then traversed in order, and each feature count is incremented by its corresponding value. Since the data is binary, this process tracks the total number of *1*s that appear for each feature under each possible

label. This produces both the total number of times each label appears and the total count of positive feature attributes associated with each possible label.

$P(y)$, the probability of each label value, is then calculated by dividing each label-value count by the total number of images trained on. $P(x_i = 1 | y)$, the probability of positive feature values for each label, is then calculated. This is done by looping through each possible label value and, for each feature, dividing the feature count, or the number of times a 1 appeared, by the number of times that label appeared.

Smoothing is applied so that no probability becomes exactly 0 or 1, since either of these values would create problems later during multiplication in prediction. To do this before division, 0.1 is added to each numerator, and 0.2 is added to each denominator. Because the data is binary, this produces usable probability values while making only a small adjustment to the original percentages.

In *naive_bayes_prediction.py*, the probabilities calculated during training are used to predict a label value for new data. A current high value is initially set to negative infinity. Each possible label value is then looped through to calculate a score based on the prior probability $P(y)$ and the conditional probabilities of the feature values for that label. If a feature in the image data is 1, the algorithm uses $P(x_i = 1 | y)$. If the feature is 0, it uses $1 - P(x_i = 1 | y)$.

To prevent underflow from repeated multiplication of small decimal values, the logarithm of each probability is used during the calculation. The logarithm-adjusted values are added together to produce a score for each possible label. The label with the highest score is then chosen as the predicted label value for that image.

Part 3b: Implement Classification Algorithms – Perceptron

In *perceptron_train.py*, the initial setup of the algorithm is the same as in *naive_bayes_train.py* through the step of determining the number of feature values. The program then initializes a 2D vector of 0s to store the weight values for each possible label. A looping variable, *weights_updated*, is set to *True*. A while loop then begins that continues until either no weight updates occur or a designated maximum number of training passes is reached.

Within the training loop, each training image is processed one at a time. For each possible label value, the feature values of the image data are multiplied by the corresponding weight values. In addition, an initial [0] weight value is included as a bias term that is independent of the image data. All of these values are summed to produce a label score.

This score is then used to check for two possible update cases. If the actual label value matches the label value currently being trained and the calculated label score is less than 0, the image feature values are added to the corresponding weight values. The bias [0] weight is also incremented by 1. The other update case occurs when the actual label value does not match the label value currently being checked, and the calculated label score is 0 or greater. In that case, the same process is applied in the opposite direction: the feature values are subtracted from the weight values, and 1 is subtracted from the bias [0] term.

This process continues for every possible label value for each training image until one full training pass has been completed. After all training images have been processed once, one training pass is complete. The loop then either continues or stops depending on whether any weight updates occurred and whether the maximum number of training passes has been reached.

In *perceptron_prediction.py*, an initial high value is set to negative infinity. The trained label weight values are then used to predict a label for new image data. For each possible label value, a score is calculated by multiplying the trained weight values by the corresponding image feature values and summing the results. The bias term is also included in this score. The label with the highest score is selected as the predicted label value.

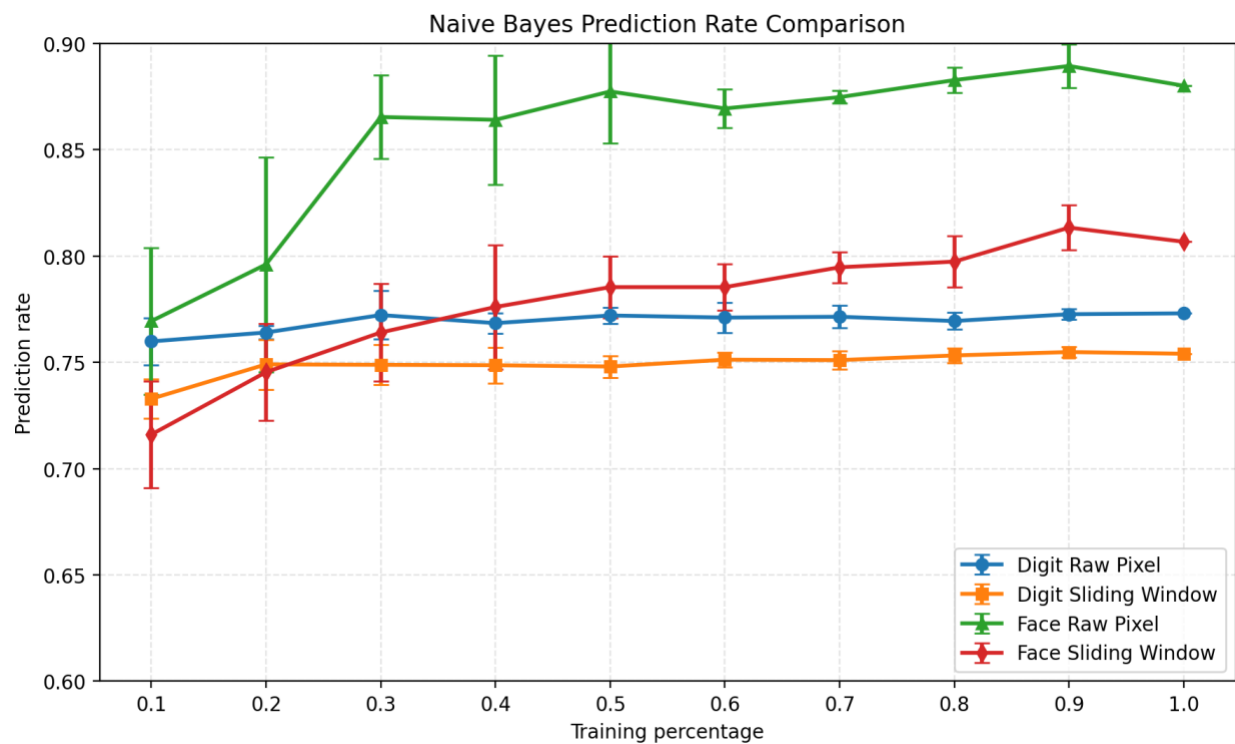
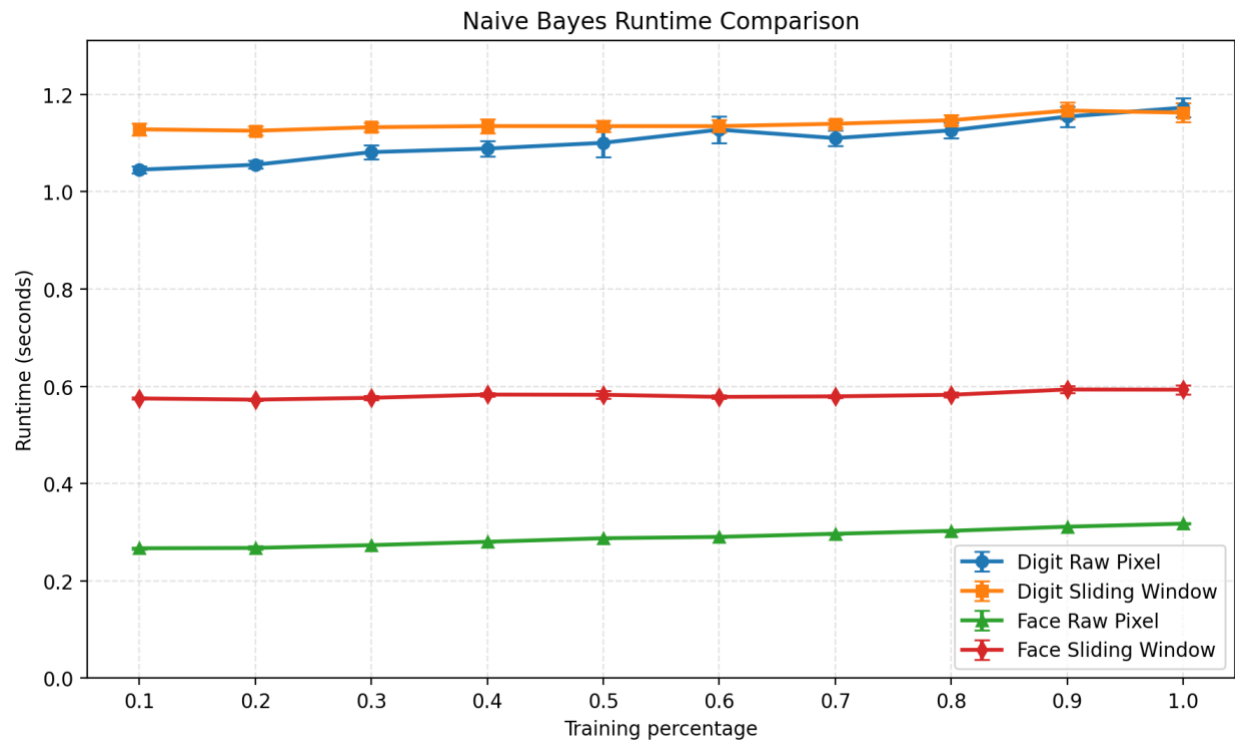
Part 4a: Algorithm Performance Across Training Percentages

The graphs below illustrate algorithm prediction rates and runtimes across training percentage levels. Runtime is measured across the full execution of each algorithm. Both prediction accuracy and elapsed runtime are then averaged across 5 runs.

Naive Bayes

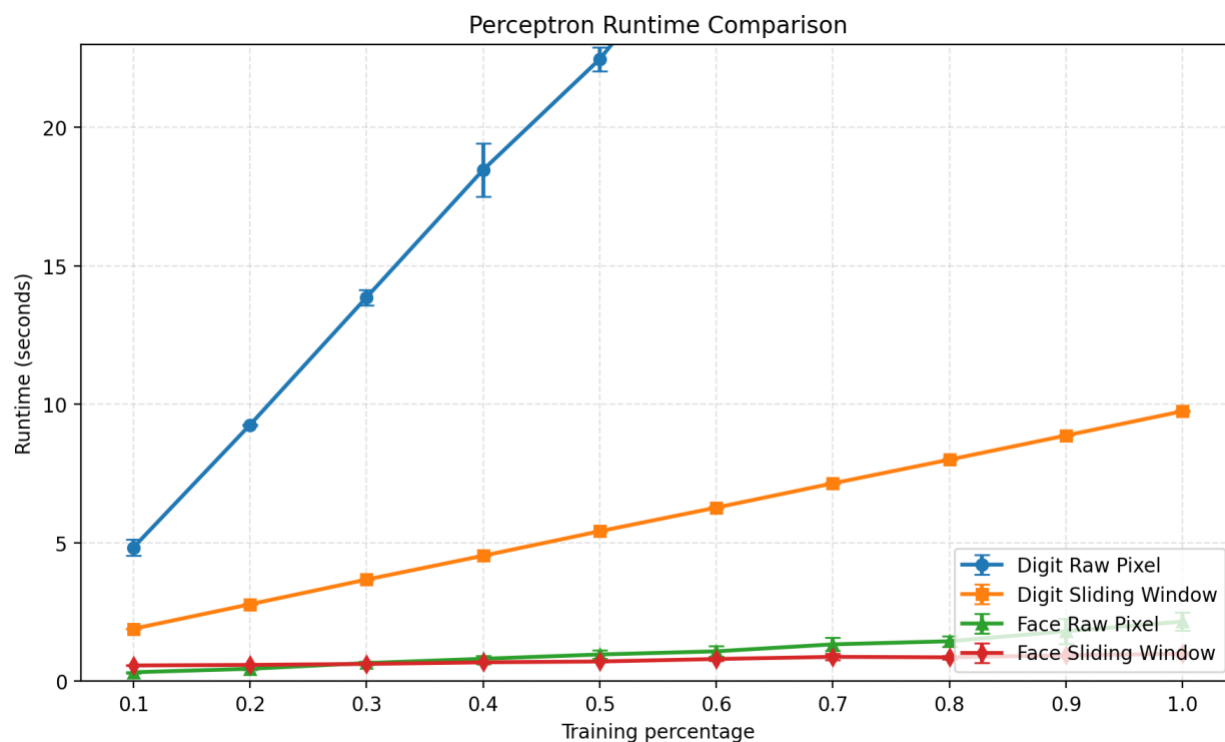
Naive Bayes runtimes remain relatively consistent across training percentages for each experiment. This is expected as the algorithm performs simple calculations during training, so increasing the amount of training data doesn't add a large amount of overhead to the total runtime. As for prediction accuracy, the face classification results show a gradual improvement as the training percentage increases, with raw pixel feature extraction performing better than sliding window. Digit classification accuracy remains fairly stable across training percentages, with no true improvement trend. Raw pixel features still perform slightly better than sliding window features for this dataset, although the difference isn't as pronounced as for the face classification. The prediction rate lines also

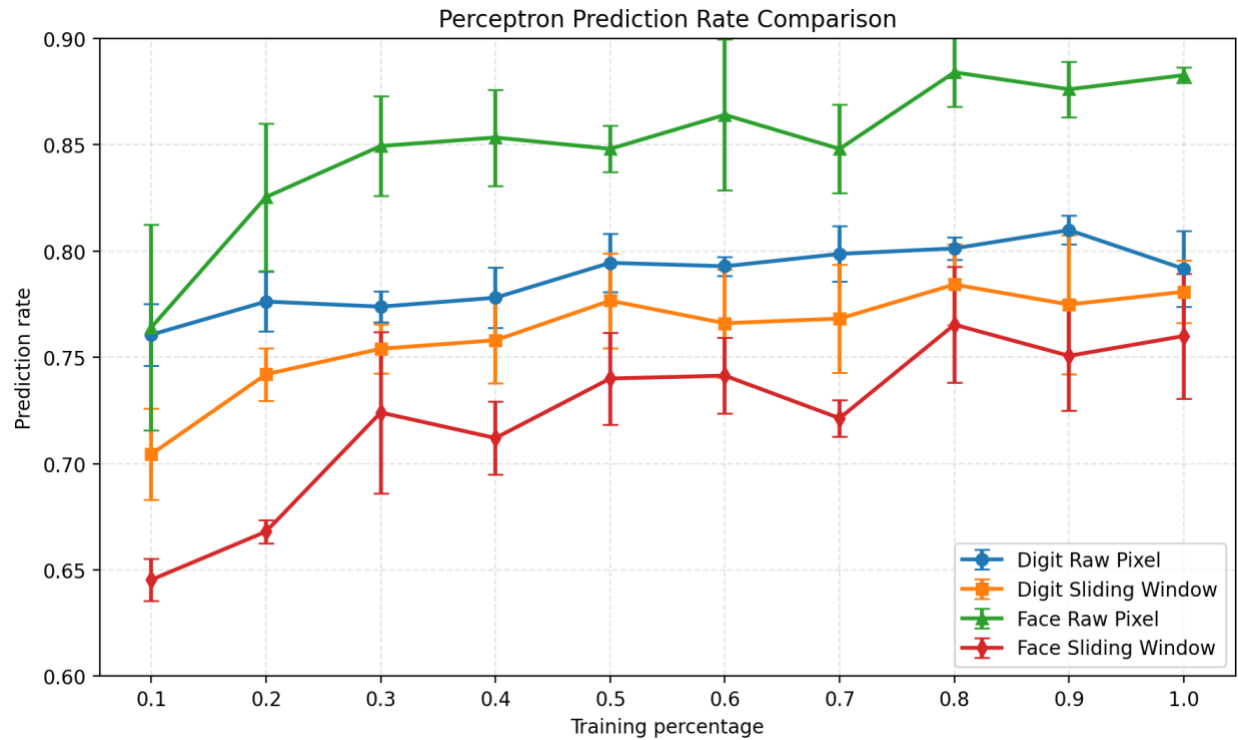
show generally smaller standard deviations as training percentages increase, indicating more consistent results as more training data is provided.



Perceptron

Perceptron runtimes show minimal increase trends for face recognition, but dramatic increases for digit recognition. Raw pixel digit recognition reached almost 45 seconds at 100% training data, placing it beyond the viewable scale of the graph. During testing, the number of training passes requiring weight adjustments was tracked, and both feature extraction methods for digit classification routinely reached the upper bound of 20 passes. This helps explain the noticeable runtime increase compared with face classification, which averaged single-digit training passes. With 10 possible label values to account for instead of 2, substantially more work is required when classifying digits over faces, leading to steep increases in runtime as more training data became available. Prediction accuracy rates show that raw pixel generally performs better than sliding window extraction, with all experiments showing some improvement as training percentages increased. Standard deviations remained high throughout all the runs, indicating Perceptron had difficulty making consistent predictions.

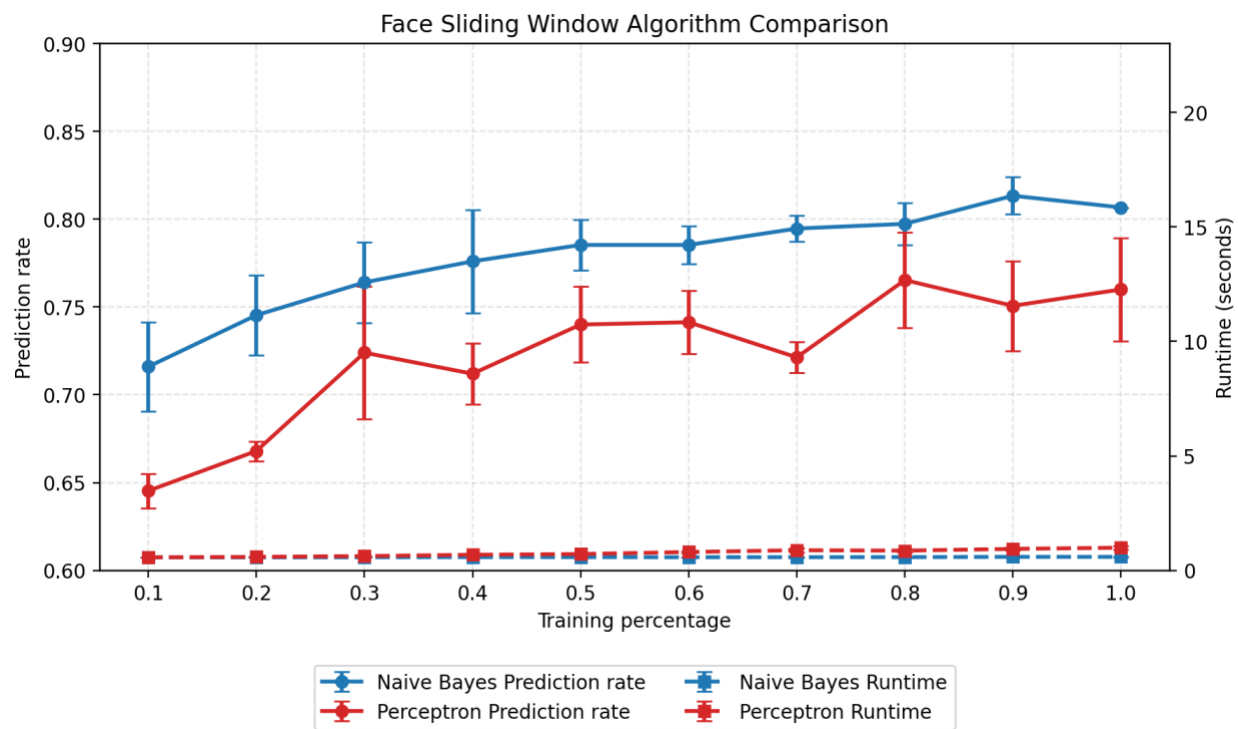
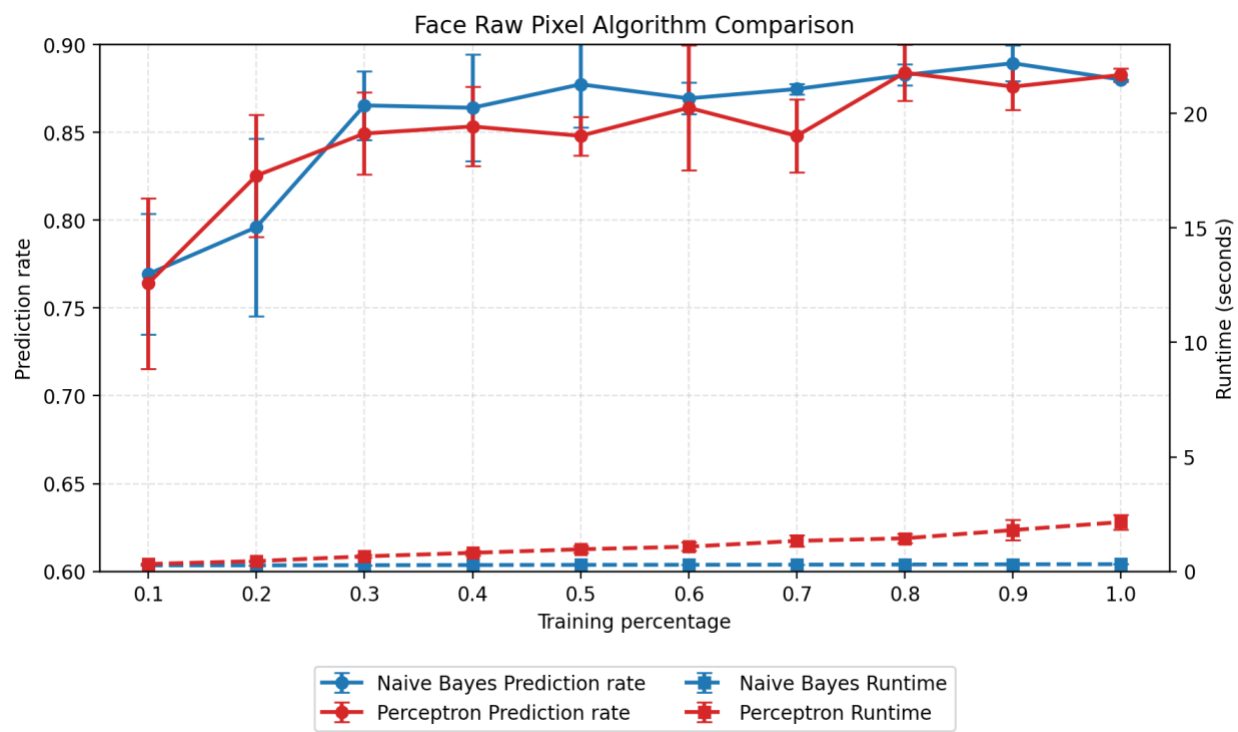


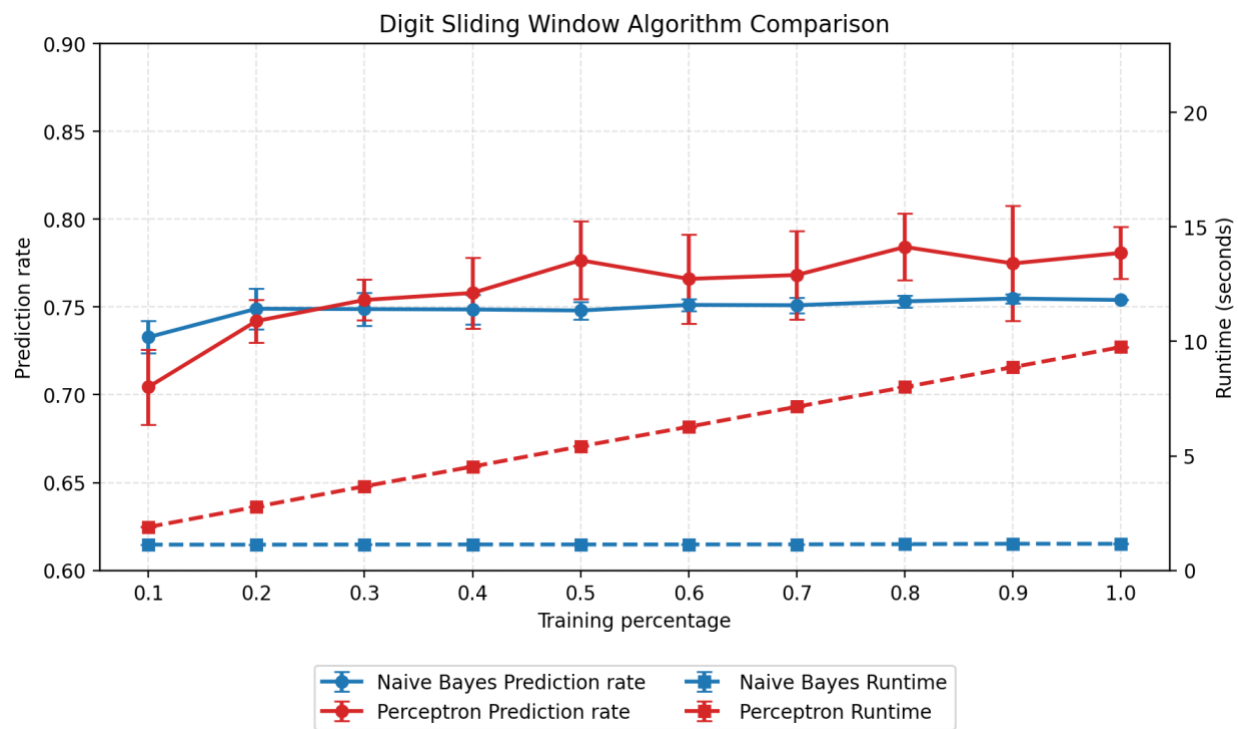
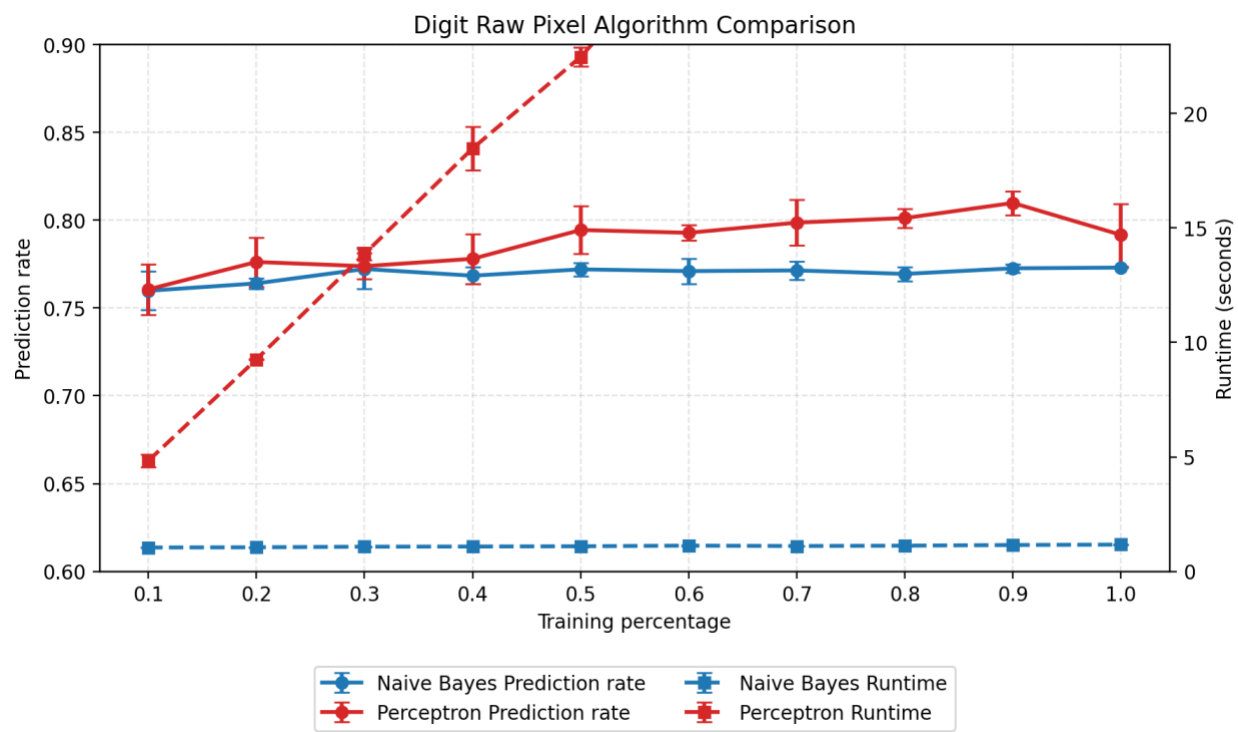


Part 4b: Algorithm Comparison Across Training Percentages

The graphs below isolate the algorithms across training percentage, feature extraction method, and image type to directly compare their performance against each other.

Analyzing the graphs shows Naive Bayes did generally better than Perceptron at face recognition, but the reverse for digit recognition. Perceptron's success in digit recognition came at a cost, though, as runtime performance jumped dramatically over that of Naive Bayes at the same training levels. Overall, their prediction rates were similar, with the largest gap showing up in face classification using the sliding window extraction method, where Naive Bayes averaged a 5 – 8 percentage point better prediction rate.





Learned Lessons and Future Improvements

I assumed Perceptron would do much better than Naive Bayes at predicting labels in all the experiments, but that wasn't the case with this simplified data. The added calculations didn't result in better results that offset the increased runtimes.

Future improvements would involve the Perceptron digit classification algorithm. It routinely maxed out the number of training passes allowed. I assume it oscillated, never finding a balance, even when I experimented with an upper bound limit of 100 passes. I attempted scaling down the weight adjustments by 0.5 and then 0.3, but found the same result.